

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА
ФРАНКА**

**Факультет прикладної математики та інформатики
Кафедра програмування**

**Лабораторна робота №4
Трансляція ASM-команд**

Виконала:
Студентка групи ПМІ-46
Тригуб Марта

Мета роботи

Ознайомитися з принципами трансляції команд мови асемблера, вивчити побудову часткового асемблера за табличним підходом та реалізувати віконну програму, що виконує двопрохідну трансляцію підмножини ASM-команд у файл формату .COM.

Постановка завдання

У роботі необхідно створити часткову програму трансляції окремих команд асемблерної програми. Під час трансляції потрібно зчитувати рядки ASM-тексту, поділяти їх на поля імені, операції та операндів, визначати спосіб адресування, будувати байт адресації, обирати шаблон машинної команди з таблиці кодів, формувати таблицю символів SYMTAB під час першого перегляду тексту програми, а під час другого перегляду підставляти адреси та записувати результат у файл .COM.

Для реалізації було обрано варіант 2 - розробку методом рефакторингу програмного коду з побудовою проєкту віконної програми. До результатів роботи також належать окремий звіт та інструкція користувача, що подаються окремими файлами від архіву проєкту.

Теоретичні відомості

Машинна команда 8086 складається з коду операції та, за потреби, додаткових полів: байта ModRM, зміщення та безпосередніх даних. Байт ModRM містить три поля: mod, reg та r/m. Значення mod визначає тип звертання до операнда, reg кодує регістр або розширення операції, а r/m задає регістр або форму адресації пам'яті.

У 16-бітному режимі для адресації пам'яті використовуються комбінації регістрів BX, BP, SI та DI. Для прямого звертання до пам'яті

за адресою або міткою застосовується спеціальний випадок $\text{mod} = 00$ та $r/m = 110$ з подальшим записом 16-бітного зміщення.

Типовий навчальний асемблер працює у два проходи. Під час першого проходу виконується аналіз рядків, обчислення довжин команд, формування таблиці символів та побудова чернеткового машинного коду з резервуванням місця під адреси. Під час другого проходу всі відкладені посилання на мітки замінюються реальними абсолютними або відносними значеннями.

Опис розробленого програмного засобу

Розроблений застосунок виконано мовою C# як WinForms-проект. Програма містить графічний інтерфейс для відкриття вихідного ASM-файлу, редагування тексту, запуску трансляції, перегляду таблиці результатів та збереження сформованого бінарного файлу.

Програму поділено на дві частини: графічну оболонку SimASM та модуль ASMEngine. Графічна оболонка відповідає за взаємодію з користувачем, а модуль ASMEngine реалізує парсинг, кодування інструкцій, формування таблиці символів, роботу з fixup-записами та запис вихідного файлу.

Структура проєкту

Файл / модуль	Призначення
Program.cs	Точка входу та запуск головної форми застосунку.
Form1.cs, Form1.Designer.cs	Графічний інтерфейс, обробка кнопок Open, Save, Compile, Compile & Save.
Settings.cs	Зберігання базових параметрів, зокрема робочої папки та режиму створення .COM.

ASMFile.cs	Основний клас асемблера, що виконує перший і другий проходи.
Parser.cs	Розбір рядка на мітку, операцію та список операндів.
Encoder8086.cs	Кодування інструкцій підмножини 8086, формування ModRM та fixup.
AsmUtil.cs	Допоміжні функції для роботи з регістрами, числами та операндами.
CompileError.cs	Клас помилок трансляції з номером рядка.
Line.cs, HexDump.cs, ExamplePrograms.cs	Сервісні класи для відображення результатів, hex-представлення та тестового прикладу.

Підтримувана підмножина мови

У межах лабораторної роботи було реалізовано підмножину команд, достатню для демонстрації двопрхідної трансляції, роботи з безпосередніми операндами, регістрами, пам'яттю та переходами.

Категорія	Елементи	Призначення
Директиви	ORG, DB, DW, END	Встановлення початкової адреси, опис даних, завершення програми.

Команди передачі даних	MOV	Пересилання між регістрами, пам'яттю та безпосередніми даними.
Арифметика і порівняння	ADD, SUB, CMP, INC, DEC	Базові арифметичні операції та порівняння.
Переходи	JMP, JE/JZ, JNE/JNZ	Безумовні і умовні переходи.
Керування	INT, NOP, RET	Виклики переривань, порожня команда, завершення.

Алгоритм роботи програми

Робота транслятора організована за двома проходами.

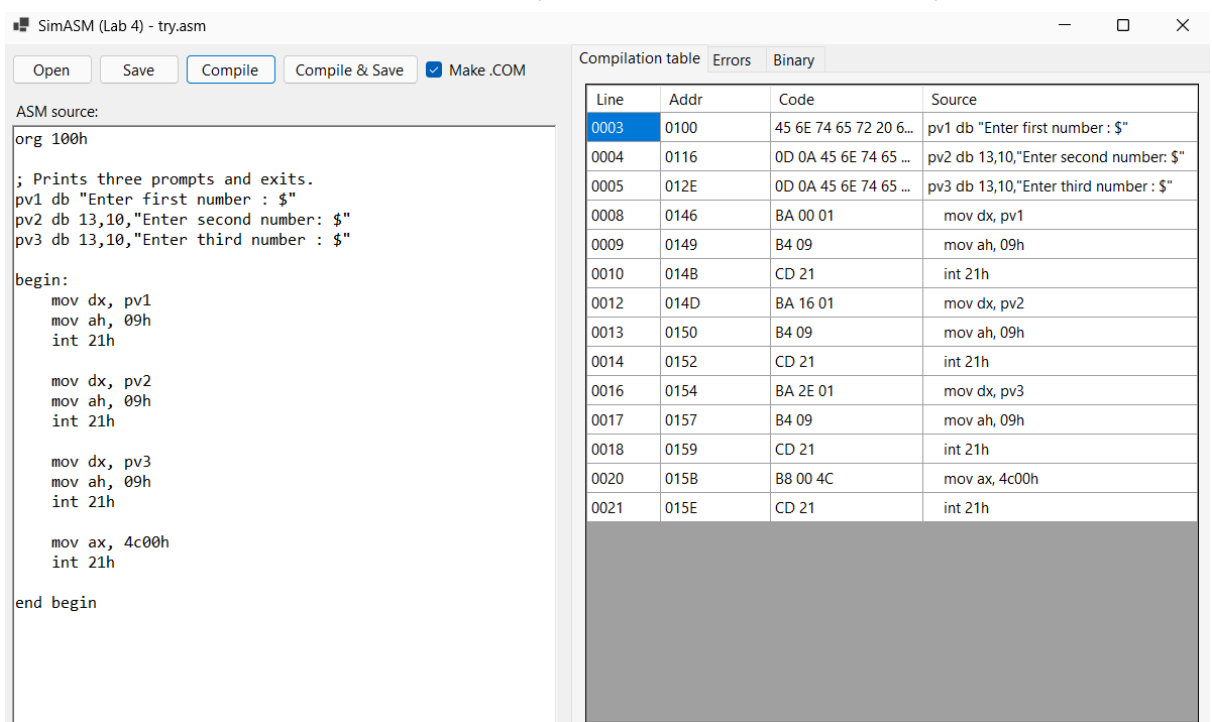
- Перший прохід: кожний рядок аналізується, мітки заносяться до SYMTAB, визначається формат команди, обчислюється довжина інструкції та формується чернетковий машинний код.
- Для команд, які містять посилання на мітки, у коді резервуються байти, а інформація про місце підстановки заносяться до списку fixup.
- Другий прохід: для кожного fixup обчислюється потрібне значення - абсолютна адреса або відносне зміщення, після чого воно записується у вихідний бінарний масив.
- Після завершення другого проходу сформований код відображається в таблиці компіляції та, за потреби, зберігається до файлу .COM.

Опис інтерфейсу користувача

Головне вікно містить редактор вихідного тексту ASM, набір кнопок керування та праву область з трьома вкладками. Вкладка Compilation

table показує номер рядка, адресу, згенерований машинний код та початковий текст. Вкладка Errors відображає повідомлення про помилки трансляції. Вкладка Binary містить hex-представлення сформованого машинного коду.

- Open - відкриття ASM-файлу.
- Save - збереження вихідного тексту програми.
- Compile - виконання трансляції без запису бінарного файлу.
- Compile & Save - трансляція з подальшим збереженням .COM-файлу.
- Make .COM - прапорець режиму створення COM-файлу.



Приклад тестування

Для перевірки працездатності застосунку використано тестову програму try.asm, яка містить директиву ORG 100h, рядки даних DB та послідовність команд MOV, INT і завершення програми. Під час компіляції програма коректно формує таблицю компіляції та зберігає результат у двійковий файл.

Результати роботи

У результаті виконання лабораторної роботи створено віконний застосунок часткового асемблера для 16-бітного режиму, який підтримує обмежену підмножину ASM-команд, реалізує двопрохідну трансляцію, формує таблицю символів SYMTAB, виконує підстановку адрес і зберігає результат у файл формату .COM.

Розроблена програма відповідає основним вимогам лабораторного завдання та може бути використана як навчальна модель побудови транслятора команд асемблера.

Висновки

Під час виконання лабораторної роботи було закріплено знання про структуру машинних команд, способи адресації в 16-бітному режимі та принципи побудови двопрохідного асемблера. Реалізація власного часткового транслятора дозволила практично застосувати табличний підхід до кодування команд і отримати навички роботи з побудовою графічного застосунку для системного програмування.

Мету роботи досягнуто: створено програму, яка виконує трансляцію підмножини ASM-команд у машинний код та записує результат у файл .COM.